	Institut Supérieur d'Informatique de Mahdia (ISIMa)
Section : GLSI **Enseignant : Mme.Waderni Sajida**	**Durée : A 2026/2027**
TP : Dév.des.app.Mobiles	

Étape 1 : Ouvrir le fichier ProductDetailsScreen.js

Ajoutez la propriété onPress au composant Pressable pour qu'il exécute une fonction lors du clic.

Modifiez le Pressable comme suit :

```
<Pressable onPress={addToCart} style={styles.button}>
        <Text style={styles.buttonText}>Add to cart</Text>
    </Pressable>
```

Étape 2 : Importer les composants nécessaires

Déclarez une fonction addToCart qui sera appelée lorsque le bouton "Add to cart" sera pressé.Ajoutez la fonction suivante en haut du composant ProductDetailsScreen

```
const addToCart = () => {
  console.warn('add to cart');
};
```

Parfait ! Pour commencer à travailler sur le composant ShoppingCart, voici une approche étape par étape.

Étape 3 : Créer le composant d'affichage des détails du produit

Créez un fichier pour le composant ShoppingCart. Par exemple, créez un fichier nommé ShoppingCart.js.

Créez le fichier cart.js dans le dossier data, si ce n'est pas déjà fait.

Copiez-collez le code suivant dans cart.js :

```
export default [
 {
  product: {
   id: "1",
   image:
    "https://notjustdev-dummy.s3.us-east-2.amazonaws.com/nike/nike1.png",
   name: "Wild Berry",
   price: 160,
  },
  size: 42,
  quantity: 2,
 },
 {
  product: {
   id: "2",
   image:
    "https://notjustdev-dummy.s3.us-east-2.amazonaws.com/nike/nike2.png",
   name: "Air Force 1",
   price: 169,
  },
  size: 43,
  quantity: 1,
```

```
  },
  {
    product: {
      id: "3",
      image:
        "https://notjustdev-dummy.s3.us-east-2.amazonaws.com/nike/nike3.png",
      name: "Nike Cosmic",
      price: 129,
    },
    size: 44,
    quantity: 1,
  },
];
```

Étape 4 :

Créer le dossier components : Dans le dossier src, créer un dossier nommé components pour organiser les composants.

Créer le fichier CartListItem.js : Dans le dossier components, créer un fichier nommé CartListItem.js.

Ajouter le code du composant :

Copier le code suivant dans CartListItem.js :

```
import { View, Text, StyleSheet, Image } from 'react-native';
import { Feather } from '@expo/vector-icons';


const CartListItem = ({ cartItem }) => {
  const increaseQuantity = () => {};
```

```
  const decreaseQuantity = () => {};

  return (
    <View style={styles.container}>
      <Image source={{ uri: cartItem.product.image }} style={styles.image} />

      <View style={styles.contentContainer}>
        <Text style={styles.name}>{cartItem.product.name}</Text>
        <Text style={styles.size}>Size {cartItem.size}</Text>

        <View style={styles.footer}>
          <Feather onPress={decreaseQuantity} name="minus-circle" size={24} color="gray" />
          <Text style={styles.quantity}>{cartItem.quantity}</Text>
          <Feather onPress={increaseQuantity} name="plus-circle" size={24} color="gray" />
          <Text style={styles.itemTotal}>${cartItem.product.price * cartItem.quantity}</Text>
        </View>
      </View>
    </View>
  );
};

const styles = StyleSheet.create({
```

```
  container: { padding: 10, paddingHorizontal: 20, flexDirection: 'row' },
  contentContainer: { flex: 1, marginLeft: 10 },
  image: { width: '40%', aspectRatio: 1 },
  name: { fontWeight: '500', fontSize: 18 },
  size: { fontSize: 16, color: 'gray' },
  quantity: { marginHorizontal: 10, fontWeight: 'bold', color: 'gray' },
  footer: { marginTop: 'auto', flexDirection: 'row', alignItems: 'center' },
  itemTotal: { fontSize: 16, marginLeft: 'auto', fontWeight: '500' },
});

export default CartListItem;
```

Installer @expo/vector-icons et react-redux :

Ouvrir le terminal et exécuter les commandes :

npm install @expo/vector-icons

npm install react-redux

Explication : Ce composant CartListItem affiche chaque article du panier avec son image, son nom, sa taille, sa quantité, et son prix, avec des boutons pour ajuster la quantité.

***Étape 5 :** Ouvrir le fichier ShoppingCart.js dans votre projet.*

Ajouter le code suivant dans ShoppingCart.js :

javascript

Copy code

```
import { FlatList } from 'react-native';
```

```
import cart from '../data/cart';
import CartListItem from '../components/CartListItem';

const ShoppingCart = () => {
  return (
    <FlatList
      data={cart}
      renderItem={({ item }) => <CartListItem cartItem={item} />}
    />
  );
};

export default ShoppingCart;
```

Composant :

Un composant est une unité réutilisable qui représente une petite partie de l'interface utilisateur, comme un bouton, une carte produit, ou dans cet exemple, un élément du panier (CartListItem).

Les composants sont conçus pour être intégrés dans plusieurs écrans ou sections de l'application.

Ils sont responsables d'une petite tâche spécifique, facilitant leur réutilisation et leur test.

screen:

Un écran (ou "screen") est une vue complète représentant une section ou une page de l'application, comme un écran de panier (ShoppingCart).

Il peut contenir plusieurs composants et est souvent utilisé pour afficher un ensemble d'éléments de l'interface.

Les écrans sont généralement connectés à la navigation de l'application et permettent de passer d'une section à une autre.

Exemple:

Le composant CartListItem affiche un produit unique dans le panier avec son image, nom, taille, et quantité.

L'écran ShoppingCart affiche la liste complète des articles du panier en utilisant plusieurs composants CartListItem.

En résumé : les composants sont des blocs de construction réutilisables, tandis que les écrans structurent l'interface globale et intègrent divers composants pour former des pages complètes de l'application.

En effet, avec React Native, il est souvent nécessaire d'installer des bibliothèques supplémentaires pour accéder à des fonctionnalités spécifiques. Voici pourquoi et comment cela diffère d'autres frameworks :

Pourquoi React Native a besoin de nombreuses bibliothèques supplémentaires

React Native fournit une base minimale pour créer des applications mobiles. Contrairement à des frameworks tout-en-un, il est conçu pour être léger et flexible, permettant aux développeurs d'ajouter seulement les fonctionnalités nécessaires via des bibliothèques tierces. Cette approche a des avantages, mais cela signifie aussi que certaines fonctionnalités de base (comme la gestion d'icônes ou le stockage d'état global) nécessitent des bibliothèques externes.

Léger et modulaire : React Native conserve une base réduite en fonctionnalités natives, préférant s'appuyer sur des bibliothèques externes pour éviter de surcharger le code de base.

Évolutivité : Les besoins peuvent varier d'une application à l'autre (par exemple, toutes les applications n'ont pas besoin de Redux pour la gestion d'état globale ou d'icônes), donc cette modularité offre une grande flexibilité.

Comparaison avec d'autres frameworks

Dans d'autres frameworks comme Flutter ou Ionic :

Flutter fournit plus de composants et de widgets dès le départ, ce qui réduit le besoin d'installer des bibliothèques externes. Par exemple, il dispose de nombreux

composants pour les animations et les icônes intégrés. Mais cela rend aussi Flutter plus volumineux, car il inclut ces fonctionnalités dans le framework lui-même.

Ionic fonctionne comme une surcouche de Cordova, ce qui signifie que certaines fonctionnalités natives nécessitent également des plugins (comme pour React Native), mais la gestion de l'état et de nombreux composants visuels sont souvent intégrés.

En résumé

Avec React Native, l'approche modulaire offre une flexibilité et des performances optimisées, mais implique de sélectionner et d'ajouter des bibliothèques pour répondre aux besoins spécifiques de chaque projet.